RAG Document Q&A Chatbot: Project Report and Interview Guide

Author: Kruthika M Gowda Stack: Python, PyMuPDF, sentence-transformers, ChromaDB, FastAPI, Streamlit, Google Gemini API

PART 1: Technical Project Report

1. Project summary

This project is a web application where a user uploads PDF documents and then asks questions about them in a chat window. The system finds the most relevant parts of the documents and uses a large language model (Gemini) to write an answer based only on those parts. Every answer shows its sources: the file name, the page number and the matching text. This technique is called **Retrieval-Augmented Generation (RAG)**.

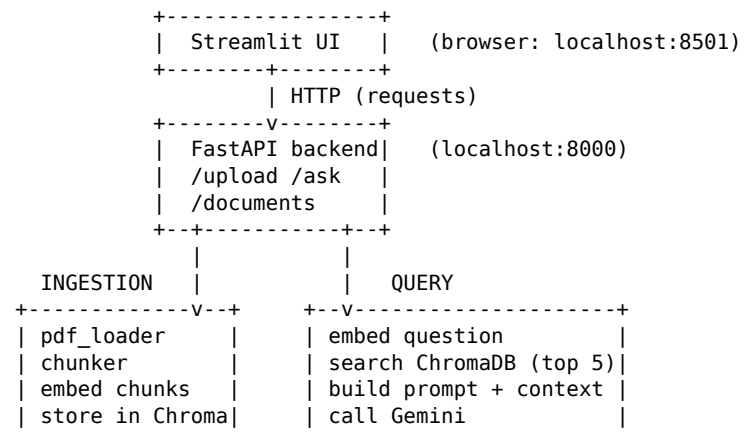
2. Problem statement

- People waste time searching long PDFs (research papers, manuals, notes, contracts).
- A plain LLM does not know the contents of private documents and may invent ("hallucinate") answers.
- Goal: an assistant that answers from the user's own documents, shows evidence, and says "I could not find it" when the answer is not there.

3. Features

- Upload one or more PDFs and process them.
- Page-aware text extraction, so citations can name the exact page.
- Chunking with overlap, local embeddings, persistent vector database.
- Semantic search (meaning-based, not just keyword matching).
- Answers with inline citations like [1], [2] and a Sources panel.
- Refuses to use outside knowledge when the context lacks the answer.
- Document list and delete from the UI.
- Retry and fallback model logic for LLM errors.
- Data persists across restarts.

4. Architecture



```
+----------------+     | return answer+sources  |
                       +------------------------+
```

Two pipelines

Ingestion pipeline (runs once per PDF):

1. Extract text page by page with PyMuPDF, keeping `source` (file name) and `page`.
2. Split each page into chunks of about 1000 characters with 200 characters of overlap.
3. Convert every chunk into an embedding vector using `all-MiniLM-L6-v2` (384 dimensions).
4. Store text, vector and metadata (source, page) in ChromaDB.

Query pipeline (runs on every question):

1. Convert the question into a vector with the same embedding model.
2. ChromaDB returns the 5 chunks with the highest cosine similarity.
3. Build a prompt: the numbered chunks (with source and page) plus the question.
4. Send it to Gemini with a system instruction: answer only from the context, cite as [n], say so if not found.
5. Return the answer and the list of sources to the UI.

5. Project structure

```
rag-chatbot/
  app/
    pdf_loader.py    PDF to page-level text
    chunker.py       overlapping chunks with ids and page numbers
    vector_store.py  embeddings, ChromaDB add/search/list/delete
    rag.py           prompt building, Gemini call, retry/fallback
    main.py          FastAPI endpoints
  ui.py              Streamlit chat UI
  data/uploads/      uploaded PDFs
  data/chroma/       persistent vector database
  .env               API key and model names
  requirements.txt
```

6. Technology choices and why

Component	Choice	Why
PDF parsing	PyMuPDF	Fast, accurate, gives text per page
Embeddings	sentence-transformers all-MiniLM-L6-v2	Free, runs locally, small (about 90 MB), good quality for its size, no API cost or rate limit
Vector DB	ChromaDB (persistent)	No server needed, simple Python API, stores metadata, cosine search
LLM	Google Gemini API	Free tier, strong at grounded question answering
Backend	FastAPI	Fast, async, automatic validation (pydantic), auto docs at /docs
UI	Streamlit	Chat interface in pure Python, quick to build

7. Key design decisions

- **Chunk per page, not per document.** Chunking each page separately means every chunk has an exact page number. This is what makes accurate citations possible.
- **Overlap of 200 characters.** A fact that sits on a chunk boundary is still fully inside at least one chunk.

- **Chunk size 1000 characters.** Large enough to hold a complete idea, small enough that the embedding stays focused and the prompt stays short.
- **Chunk ids like `file.pdf-p5-c2`.** Re-uploading a file replaces old chunks (the app deletes the old ones first) instead of creating duplicates.
- **Cosine similarity.** Compares the direction of vectors, so text length matters less.
- **Top-k = 5.** A balance between giving enough context and not flooding the LLM.
- **Temperature 0.2.** Low randomness for factual answers.
- **System prompt that forbids outside knowledge.** Reduces hallucination and makes the bot honest when documents lack the answer.
- **Same embedding model for documents and questions.** Required, otherwise the vectors are not comparable.

8. API endpoints

Method	Path	Purpose
GET	/	Health check
POST	/upload	Upload PDFs, extract, chunk, embed, store
POST	/ask	Body: question and k. Returns answer and sources
GET	/documents	List stored documents with chunk counts
DELETE	/documents/{name}	Delete a document's chunks and file

9. Problems faced and how they were solved (good interview material)

Problem	Cause	Solution
Script printed nothing	Files were created empty	Pasted the code into each file
`FileNotFoundError` / 0 pages	File was `sample.pdf.docx`, then a scanned PDF	Used a real text-based PDF; learned scanned PDFs need OCR
`ModuleNotFoundError: google.genai` / `fastapi`	Packages not installed in the venv	Recreated the venv and ran `pip install -r requirements.txt`
404 NOT_FOUND on model	Model `gemini-2.5-flash` retired for new users	Made the model name configurable in `.env` and listed available models through the API
Streamlit `ReadTimeout`	Backend stalled while running the embedding model inside a worker thread	Made endpoints `async` and preloaded the model at startup (lifespan)
Timeouts on the 2nd question	UI timeout (120 s) shorter than the LLM retry time	Aligned timeouts and added logging of each step
503 UNAVAILABLE	Gemini overloaded	Added retry and a fallback model
429 RESOURCE_EXHAUSTED	Free tier limit of 20 requests/day per model was used up	Switched models, stopped retrying on 429, planned a new key or billing
Tooling confusion	Two terminals, nested folders, OneDrive	Moved the project to `C:\projects`, one folder, two clearly named terminals

10. Limitations and future improvements

- Scanned PDFs need OCR (Tesseract or similar).
- Single shared document collection, with no user accounts.
- No conversation memory: each question is independent.
- Fixed-size character chunking; semantic or section-aware chunking would be better.
- Retrieval is vector-only. Hybrid search (keyword BM25 plus vectors) and a re-ranker would improve accuracy.
- No automated tests, Docker or deployment yet.
- Free-tier LLM quotas limit usage.
- Answers are not streamed token by token.

11. Possible next steps

1. Authentication (JWT) and per-user collections.
2. Chat history and follow-up questions (query rewriting).
3. Hybrid search and a cross-encoder re-ranker.
4. OCR for scanned documents; support for DOCX and web pages.
5. Evaluation set to measure answer quality (for example with RAGAS).
6. Tests with pytest, Docker, and deployment on Render or Railway.
7. Streaming answers.

12. How to run

```
cd C:\projects\rag-chatbot
venv\Scripts\activate
uvicorn app.main:app            (window 1, keep open)

cd C:\projects\rag-chatbot
venv\Scripts\activate
streamlit run ui.py             (window 2, keep open)
```

Open `localhost:8501`. Upload a PDF, click Process files, ask questions.

PART 2: HR / Non-Technical Explanation

The 30-second version

"I built an AI assistant that reads your documents and answers questions about them. You upload a PDF, ask a question in plain English, and it gives you the answer along with the exact page it found it on, so you can verify it. If the answer is not in the document, it tells you instead of making something up."

A simple analogy

Imagine a very fast librarian. When you give them a 300-page report, they read it and index every section. When you ask a question, they do not read the whole book again. They go straight to the 5 most relevant paragraphs, read those, and write you a clear answer, pointing to the page numbers. That is exactly what this system does.

Why it matters (business value)

- **Saves time:** finding information in long documents takes seconds instead of hours.
- **Trustworthy:** every answer has a source, so people can check it.
- **Honest:** it refuses to guess when the document has no answer, which reduces the risk of wrong information.
- **Private data friendly:** it works on a company's own documents, which a general chatbot has never seen.
- **Reusable:** the same idea works for HR policies, legal contracts, customer support manuals, research papers, student notes and onboarding documents.

What it shows about me

- I can build a complete product end to end: backend, database, AI, and user interface.
- I used current AI engineering practice (RAG), not just calling a chatbot.
- I solved real problems: API limits, outages, timeouts and environment issues, and I documented them.
- I make design choices on purpose and can explain the trade-offs.
- I can learn new tools quickly: FastAPI, vector databases and LLM APIs.
- I take ownership: I debugged systematically, using logs and isolated tests, instead of guessing.

Typical HR questions and answers

"Tell me about this project." "It is a document question-answering chatbot. Users upload PDFs and ask questions. The system searches the documents, finds the most relevant passages and uses an AI model to write an answer with page citations. I built the whole thing in Python."

"What was the hardest part?" "Reliability. The AI model sometimes returned overload or quota errors, and my app timed out. I added logging to find where it stalled, fixed how the server ran the model, added retries and a backup model, and learned to handle free-tier limits. It taught me that a working demo and a dependable system are different things."

"Did you work alone? What was your role?" "I built it myself: design, coding, testing and debugging. I used documentation and AI tools to learn faster, and I made sure I understood every part."

"What would you improve with more time?" "User login, support for scanned documents, better search accuracy, automated tests and deployment so others can use it online."

"Why should we care?" "Companies have huge amounts of internal documents. Tools like this make that knowledge searchable and usable. I understand how to build one and how to make it reliable."

PART 3: RAG Explained in Detail

1. What is RAG?

Retrieval-Augmented Generation combines two steps:

1. **Retrieval:** find the pieces of information relevant to the question from an external knowledge source (your documents).
2. **Generation:** give those pieces to a language model as context so it writes an answer grounded in them.

Instead of relying on what the model memorized during training, the model "reads" the relevant text at question time.

2. Why RAG is needed

LLMs have limits:

- **Knowledge cutoff:** they do not know recent events.
- **No access to private data:** they have never seen your company documents.
- **Hallucination:** they can produce confident but false answers.
- **Limited context window:** you cannot paste a whole library into the prompt.
- **No citations:** a plain LLM cannot show where an answer came from.

RAG addresses all of these by fetching only the relevant text and attaching it to the prompt.

3. RAG vs fine-tuning

	RAG	Fine-tuning
What changes	The prompt (context is added)	The model's weights
Updating knowledge	Upload a new document, immediately available	Retrain, slow and costly
Citations	Easy	Hard
Hallucination control	Good (grounded)	Weaker
Best for	Facts, documents, changing data	Style, tone, specialized behavior
Cost	Low	High

They can be combined, but RAG is usually the first choice for question answering over documents.

4. The RAG pipeline step by step

A. Loading

Read the source files. Here: PyMuPDF extracts text per page. Scanned PDFs need OCR because they have no text layer.

B. Chunking

Split text into smaller pieces because:

- Embedding models and LLM prompts have size limits.
- A small, focused chunk gives a more precise match than a whole document.

Trade-offs:

- **Too small:** loses context, the answer is split across chunks.
- **Too large:** the vector becomes a blurry average, and retrieval gets less precise and prompts get expensive.

- **Overlap:** repeated text between neighbors, so ideas at the boundaries are not cut in half.

Other strategies: by sentence, by paragraph, by heading or section, recursive splitting, and semantic chunking.

C. Embeddings

An embedding model turns text into a vector (a list of numbers, here 384) so that texts with similar meaning get vectors that are close together.

- "How do I reset my password?" and "Steps to change login credentials" are close, even though they share few words.
- This is why semantic search beats keyword search for natural questions.
- Documents and questions must use the **same** embedding model.

D. Vector database

Stores the vectors with the text and metadata and finds the nearest vectors quickly using approximate nearest neighbor (ANN) algorithms such as HNSW. Examples: ChromaDB, FAISS, Pinecone, Weaviate, Qdrant, pgvector. This project uses ChromaDB with cosine distance.

E. Retrieval

Embed the question, then fetch the top-k closest chunks.

- **Similarity measures:** cosine similarity (angle between vectors), dot product, Euclidean distance.
- **top-k:** how many chunks to return. Too few can miss the answer, too many adds noise and cost.

F. Prompt construction (augmentation)

Combine instructions, the retrieved chunks (numbered, with source and page) and the question. Instruct the model to use only the context, cite by number and admit when the answer is missing.

G. Generation

The LLM writes the answer from the context. A low temperature (0.2) keeps it factual.

H. Citations

Because each chunk carries metadata (file, page), the app can show where each statement came from, which builds trust and allows verification.

5. Important concepts to know

- **Token:** a piece of text (roughly 3 to 4 characters) that LLMs read and are billed on.
- **Context window:** the maximum number of tokens the model can read in one request.
- **Hallucination:** fluent but unsupported output. Grounding, citations and "say you do not know" instructions reduce it.
- **Semantic search vs keyword search:** meaning-based vs exact-word matching.
- **Hybrid search:** combine BM25 keyword scores with vector scores, often with better results.
- **Re-ranking:** retrieve more (say 20) and re-order with a cross-encoder model that scores each question and chunk pair more accurately, then keep the best few.
- **Query rewriting:** rewrite follow-up questions ("what about its limitations?") into standalone queries using chat history.
- **Metadata filtering:** restrict search to a document, user or date.
- **Lost in the middle:** LLMs may pay less attention to the middle of long contexts, so ordering and limiting context matters.

- **Prompt injection:** malicious text inside a document that tries to give the model instructions. Treat retrieved text as data, not instructions.

6. Evaluating a RAG system

Measure the two stages separately:

- **Retrieval quality:** recall@k (is the right chunk in the top k?), precision@k, MRR.
- **Answer quality:** faithfulness (is the answer supported by the context?), answer relevance, context relevance, correctness against reference answers.
- Tools and methods: RAGAS, TruLens, LLM-as-a-judge, a hand-built test set of question and answer pairs.

7. Common RAG failure modes and fixes

Failure	Cause	Fix
Right answer exists but is not found	Poor chunking, weak embeddings	Tune chunk size and overlap, hybrid search, better embedding model
Answer mixes unrelated facts	Too many or noisy chunks	Lower k, re-rank, filter
Hallucinated answer	Weak prompt, irrelevant context	Strict system prompt, "say not found", lower temperature
Misses tables or figures	PDF text extraction loses structure	Table-aware parsers, OCR, multimodal models
Slow responses	Big prompt, model load, cold start	Smaller k, preload model, caching, streaming
Follow-up questions fail	No memory	Chat history and query rewriting

PART 4: Interview Questions and Answers

A. Project questions

1. Explain your project in 1 minute. "It is a RAG-based document Q&A chatbot. A user uploads PDFs; I extract text page by page, split it into overlapping chunks, embed the chunks with a sentence-transformers model and store them in ChromaDB. When the user asks a question, I embed it, retrieve the top 5 similar chunks, and send them with the question to Gemini with instructions to answer only from the context. The response is shown with citations: file name and page number. The backend is FastAPI and the UI is Streamlit."

2. Why RAG instead of just asking the LLM? The LLM does not know private documents, may hallucinate, and cannot cite sources. RAG supplies relevant text at question time, gives grounded answers and enables citations.

3. Why did you chunk by page? To keep an exact page number on every chunk for citations.

4. How did you choose chunk size and overlap? 1000 characters (a paragraph or two) with 200 overlap, as a common starting point. It keeps ideas intact without making embeddings too diluted. I would tune it using an evaluation set.

5. Why sentence-transformers and not an API embedding model? It is free, runs locally, has no rate limits, keeps data on the machine, and `all-MiniLM-L6-v2` is small and good enough. The downside is a smaller model than top commercial embedders and a startup load time.

6. Why ChromaDB? Embedded, persistent, zero server setup, simple API, metadata support. For production scale I would consider pgvector, Qdrant or Pinecone.

7. What similarity metric did you use? Cosine similarity, configured with `hnsw:space = cosine`. I report score as 1 minus the cosine distance.

8. How do you prevent hallucinations? A system prompt that restricts answers to the numbered context, requires citations, and instructs the model to say it could not find the answer; low temperature; showing sources so users can verify.

9. How do citations work? Each chunk is stored with `source` and `page` metadata. The prompt labels chunks [1] to [5]. The model cites those numbers, and the UI maps them to file and page and shows a snippet.

10. What happens if the same PDF is uploaded twice? The app deletes the old chunks of that file first and then inserts the new ones. Chunk ids are deterministic (`file-p5-c2`), and I use upsert.

11. What if a PDF is scanned? PyMuPDF returns no text, and the app reports "No extractable text". The fix is OCR with Tesseract, which is a listed improvement.

12. How did you handle LLM errors? Retry on temporary errors (503), fallback to a second model, no retry on 429 (quota), timeouts on the HTTP client, and clear error messages shown in the UI.

13. What was your hardest bug? The app timing out after the first question. I isolated each component: tested Gemini alone, then search alone, then the whole pipeline in a script, which worked. That showed the problem was in how the server ran it. I preloaded the model at startup and made the endpoints async. Later, logging showed the real causes were Gemini 503 and 429 quota errors, so I added a fallback model and stopped retrying on 429.

14. Why `async def` fixed it? (be honest) My isolated test ran in the main thread and worked, while a plain `def` endpoint runs in a worker thread, where the PyTorch model stalled on Windows. Making the endpoint async ran it on the main event loop. The trade-off is that a long synchronous call blocks the event loop; for production I would use a worker process or a dedicated embedding service.

15. What are the scalability limits? Single process, local embedding on CPU, file-based Chroma, synchronous LLM call, and a shared collection. To scale: separate embedding service, hosted vector DB, background ingestion queue (Celery), caching, multiple workers and per-user collections.

16. How would you add chat memory? Keep the history, rewrite each follow-up into a standalone query with the LLM, retrieve using the rewritten query, and include recent turns in the prompt.

17. How would you improve retrieval accuracy? Hybrid BM25 and vector search, re-ranking with a cross-encoder, better chunking (by section), larger k then rerank, metadata filters, a stronger embedding model, and evaluation to tune.

18. How would you secure it? Authentication, per-user data isolation, file type and size validation, hide API keys in environment variables (never commit `.env`), rate limiting, and guard against prompt injection from documents.

19. How would you test it? Unit tests for the chunker (overlap, boundaries, empty text), API tests with FastAPI TestClient and a mocked LLM, retrieval tests on a small fixed document, and an evaluation set for answer quality.

20. How would you deploy it? Dockerize backend and UI, store secrets as environment variables, mount a persistent volume for Chroma, deploy on Render or Railway or a VPS, and add a reverse proxy with HTTPS.

B. Python and backend questions

What is FastAPI and why use it? A modern Python web framework built on Starlette and Pydantic. It gives high performance, async support, automatic request validation and automatic interactive docs at `/docs`.

`def` **vs** `async def` **in FastAPI?** `def` endpoints run in a thread pool; `async def` runs on the event loop and should not block. Use `async` with awaitable I/O, and `def` for blocking code, unless you offload it.

What is a virtual environment? An isolated Python environment with its own packages, so projects do not conflict.

What is `.env` **and why?** A file of configuration and secrets loaded into environment variables (python-dotenv). It keeps keys out of code and out of Git (`.gitignore`).

What is Pydantic? A library that validates data using type hints. `AskRequest` validates that `question` is a string and `k` is an integer.

What is a REST API? An HTTP interface using methods (GET, POST, DELETE) and URLs to act on resources, usually exchanging JSON.

HTTP status codes seen here: 200 OK, 400 bad request, 404 not found, 429 too many requests (quota), 500 server error, 503 service unavailable.

What is a timeout and why set one? A limit on how long to wait for a response, so a stuck dependency fails with a clear error instead of hanging forever.

What is a Streamlit session state? A dictionary that survives reruns, used to keep the chat history.

C. Machine learning and NLP questions

What is an embedding? A dense numeric vector representing meaning; similar texts have nearby vectors.

How is `all-MiniLM-L6-v2` **trained?** It is a small transformer (6 layers) fine-tuned with contrastive learning on sentence pairs so that related sentences are close; it outputs 384 dimensions and has a 256-token input limit by default (longer text is truncated), which is another reason to chunk.

What is the Transformer? The architecture from "Attention Is All You Need" (the sample PDF used in testing): it replaces recurrence with self-attention, processes tokens in parallel, uses positional encodings, and underlies modern LLMs.

Self-attention in one line: each token computes weights over all other tokens (using queries, keys and values) to build a context-aware representation. Scaled dot-product attention is

softmax(QK^T / sqrtd_k) V.

Cosine similarity formula: cos(a, b) = (a · b) / (||a|| ||b||). Ranges from -1 to 1; closer to 1 means more similar.

What is HNSW? Hierarchical Navigable Small World graph: an approximate nearest neighbor index that gives fast searches with high recall.

Precision vs recall in retrieval? Precision: of the retrieved chunks, how many are relevant. Recall: of all relevant chunks, how many were retrieved.

What does temperature do? Controls randomness in token sampling. Low values give more deterministic, factual output.

What is a system prompt? Instructions that define the model's role and rules, separate from the user's question.

What are guardrails? Rules and checks that keep output safe and on topic, such as refusing out-of-scope questions.

D. Scenario and behavioral questions

The bot gives a wrong answer. How do you debug?

1. Check the retrieved chunks. If the right chunk is missing, it is a retrieval problem (chunking, embeddings, k).
2. If the right chunk is there but the answer is wrong, it is a generation problem (prompt, model, temperature).
3. Fix, then re-run an evaluation set to confirm nothing else got worse.

The app is slow. What do you check? Model loading time (preload), embedding time, size of k and prompt, LLM latency, network, and whether work is blocking the event loop. Add timing logs for each step, as I did.

What did you learn? How RAG works end to end, how to debug by isolating components, handling third-party API failures and quotas, and that configuration (model names, keys, environment) is a major source of real-world bugs.

Tell me about a time you were stuck. Use the timeout story above: describe the symptoms, how you narrowed it down with isolated tests and logs, the root causes and the fixes.

Why did you pick this project? RAG is one of the most common real-world LLM applications, and it covers a full stack: data processing, ML, backend, frontend and operations.

PART 5: Quick Revision Cheat Sheet

- **RAG = Retrieve relevant chunks, then Generate an answer from them.**
- Pipeline: load, chunk, embed, store, (question) embed, retrieve top-k, prompt, generate, cite.
- Settings used: chunk 1000, overlap 200, top-k 5, cosine, 384-dim embeddings, temperature 0.2.
- Why chunking: size limits and precise retrieval. Why overlap: do not cut ideas at boundaries.
- Why the same embedding model for both documents and queries: vectors must share one space.
- Hallucination control: grounded prompt, citations, low temperature, "say not found".
- Errors: 404 wrong model, 429 quota, 503 overload, ReadTimeout slow or stuck backend.
- Improvements: hybrid search, re-ranking, chat memory, OCR, auth, tests, Docker, deployment, evaluation.

Security note

Never share or commit your `.env` file. If an API key appears in a screenshot or chat, delete it in Google AI Studio and create a new one.